

A3-PS023-TC389

Minimal Two-Node Cloud Hypervisor VM Controller

Dr Om Prakash Singh **Harsh Kharwar** **Yug Sharma**

Cod-i

AIORI Hackathon 3 · Category: Cloud Computing

October 14, 2026

Problem Description

Enterprise hypervisor stacks (OpenStack, libvirt) run many daemons, databases and queues – too heavy for a two-host edge site.

We build a **minimal controller + two worker agents** that create, start, stop and list VMs on two `/dev/kvm` hosts through the **Cloud Hypervisor REST API**.

- **Placement** – pick a host by free memory without over-commit or stranded capacity
- **Security** – an open control channel equals remote code execution on the host
- **Limits** – one VM must never starve the host or its neighbours
- **Leaks** – a failed API call can leave orphan VMM processes, sockets, TAP devices and pinned memory
- **Crashes** – a dead VM must be detected and healed, not silently forgotten

Out of scope (per PS): live migration, distributed storage, browser VNC, image marketplace, writing a new hypervisor.

Background

KVM + Cloud Hypervisor: one small process per VM, driven over a Unix socket

- **KVM** turns Linux into a type-1 hypervisor; `/dev/kvm` exposes CPU virtualisation to user space.
- **Cloud Hypervisor** is a Rust VMM on KVM with only modern virtio devices – small attack surface, fast boot.
- Each VM is one `cloud-hypervisor` process started with `--api-socket`; it is controlled with HTTP/JSON over that socket.
- No daemon of its own – someone must own the process lifecycle. That is our agent.

Our operation	Cloud Hypervisor calls
create	spawn VMM → `PUT vm.create`
start	`PUT vm.boot`
stop	`PUT vm.shutdown`
list	`GET vm.info`, `GET vmm.ping`
delete	`PUT vm.delete` → `PUT vmm.shutdown` → reap PID

All endpoints live under `/api/v1/` on the per-VM Unix socket [1].

Literature & RFC Review

What we borrow, and from where

Source	What we take from it
Cloud Hypervisor API [1]	VM config schema, lifecycle endpoints, per-VM API socket
Kubernetes architecture & scheduler [2,3]	Declarative state + reconcile loop; node agent; filter → score scheduling; balanced-allocation scoring
etcd / Raft [4]	Consistent store with transactions (compare-and-swap), watches and leases
Firecracker, NSDI '20 [5]	One process per microVM, cgroup confinement, minimal device model
Sagas, SIGMOD '87 [6]	Multi-step operation = steps + compensating actions → transactional rollback
RFC 8446 (TLS 1.3), RFC 7519 (JWT), RFC 2104 (HMAC) [7–9]	Mutual TLS between controller and agents; signed, short-lived tokens
RFC 9110 (HTTP semantics) [10]	Idempotent `PUT`/`DELETE`, status codes for API errors
Linux cgroup v2 [11]	`memory.max`, `cpu.max`, `pids.max` for hard per-VM limits

Proposed Solution

A Kubernetes-style control plane, cut down to what two hosts need

Control plane (runs on Host A)

API service – user-facing REST endpoint; authentication, input validation, limit admission.

Scheduler – picks a host for each pending VM (filter, then weighted score).

Controller – reconcile loop: compares desired with actual state; drives create/stop, rollback and auto-heal.

etcd – persistent store for VM specs, node heartbeats and memory reservations.

Worker plane (Host A and Host B)

Node agent (kubelet-style) – accepts signed requests from the controller and calls the Cloud Hypervisor API.

Monitor – separate process; watches VMM PIDs and polls `vmm.ping`/`vm.info`; reports health.

Garbage collector (GC) – kills orphan VMMs and frees their memory, sockets, TAP devices and cgroups.

Proxy – VM service networking (future scope).

User API: `POST /v1/vms` · `POST /v1/vms/{id}/start` · `POST /v1/vms/{id}/stop` · `GET /v1/vms` · `DELETE /v1/vms/{id}`

Optimization Proposed

Placement: filter, then a weighted score

1. Filter – drop a host if

- heartbeat older than 10 s (NotReady)
- free memory − host reserve < VM memory + VMM overhead
- free vCPU < requested vCPU

2. Score the survivors (memory first, per PS):

$$S = 0.60\,M' + 0.25\,C' - 0.15\,F$$

M' = free memory after placement / allocatable
C' = free vCPU after placement / allocatable
$F = |M' - C'|$ = imbalance, i.e. capacity left stranded

3. Commit – reserve memory with an etcd transaction (compare revision): two concurrent requests cannot double-book a host.

Worked example – request: 2 GiB, 2 vCPU
Both hosts: 16 GiB (15 allocatable), 8 vCPU

	Host A	Host B
Used before	9 GiB / 2 vCPU	8 GiB / 6 vCPU
Free after	4 GiB / 4 vCPU	5 GiB / 0 vCPU
M', C', F	0.27, 0.50, 0.23	0.33, 0.00, 0.33
Memory-only	–	**picked**
Weighted S	**0.25 picked**	0.15

Memory-only placement sends the VM to B, leaving 5 GiB stranded with zero vCPU. The weighted score keeps both hosts usable.

Optimization Proposed

Baseline vs. our design

Aspect	Naive controller	Our design
Footprint	Many daemons + SQL DB + message queue	One controller binary + etcd; one agent + monitor per host
Placement	First-fit or most-free-memory	Filter + weighted score (memory, vCPU, fragmentation)
Concurrency	Races can double-book memory	Reservation committed in an etcd transaction
Failure mid-create	Orphan process, socket, TAP device	Saga rollback: every step has a compensating action
VM crash	Stays dead; state goes stale	Monitor detects it; controller restarts it with backoff
Leaked resources	Manual clean-up	Periodic GC + reconcile on agent start
Control channel	Open HTTP / shared Unix socket	mTLS + signed, short-lived requests; sockets never on the network

Solution Architecture & Design

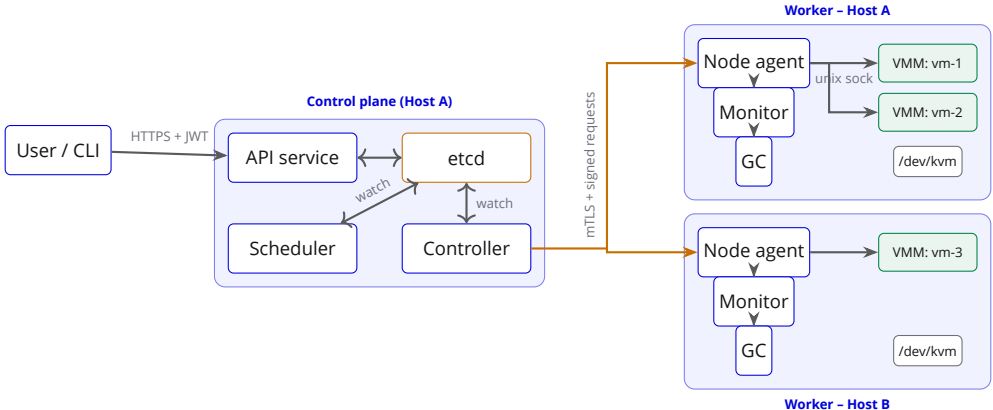


Figure: System architecture – solid orange: authenticated controller↔agent channel

Data Flow Diagram

Create VM: happy path and rollback

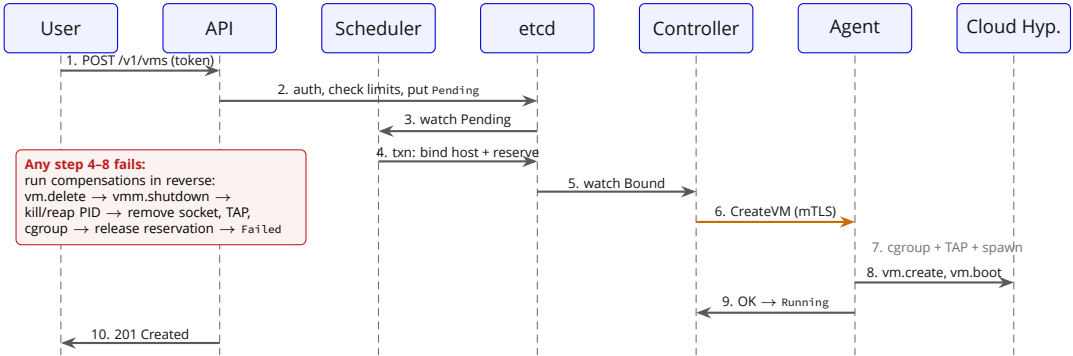


Figure: Create-VM sequence (start, stop and delete follow the same controller → agent → VMM path)

Failure Handling & Clean-up

No VM process outlives its record; no record outlives its VM

Transactional rollback

Step	Undo
reserve mem	release
cgroup	rmdir
TAP device	ip link del
spawn VMM	shutdown, kill
vm.create	vm.delete
vm.boot	vm.shutdown

Each step is journaled on the agent before it runs, so a crashed agent finishes the rollback on restart.

Garbage collector

Runs every 30 s and on agent start.

Lists VMM PIDs in the agent's cgroup slice and sockets in `/run/vmctl`, and compares them with etcd.

Orphan → `vmm.shutdown` → SIGTERM → SIGKILL → reap → free memory, socket, TAP, cgroup.

Fail-safe: if the controller is unreachable, GC kills nothing.

Monitor + auto-heal

Monitor gets VMM exit at once via `pidfd`, and polls `vmm.ping`/`vm.info` every 5 s.

Desired `Running`, actual dead → controller restarts it on the same host (backoff 1, 2, 4 s …).

Host NotReady > 30 s → cold-boot on the other host from the prepared image (not live migration).

A generation number makes the stale copy get reaped when the old host rejoins.

Resource Limits & Authentication

Resource-limit enforcement

Admission (API): 1–4 vCPU and 256 MiB–4 GiB per VM (configurable); max VMs per host.

Host reserve: 1 GiB kept for the host OS; no memory over-commit.

Runtime (agent): one cgroup v2 per VM

- `memory.max` = guest + VMM overhead
- `memory.swap.max` = 0
- `cpu.max` = vCPU × period
- `pids.max` caps thread count

Double check: agent re-reads `MemAvailable` before spawning and refuses if the scheduler's view was stale.

Authenticated API

User → API: HTTPS (TLS 1.3) + short-lived JWT with role and expiry.

Controller ↔ agent: mutual TLS with a private CA (one cert per host). Each request also carries an HMAC-SHA256 signature over method, path, body, timestamp and nonce.

Rejected: wrong certificate, clock skew $>$ 30 s, replayed nonce → `401`.

Agent → VMM: Unix sockets in a `0700` directory owned by the agent; never exposed on the network.

Agent listens only on the management interface.

Test Plan

Lifecycle, two-host placement and failure injection

Lifecycle

create → start → list → stop → start → delete; check `vm.info` state after each step.

Repeat stop/delete: idempotent, no error.

Reject out-of-limit requests at admission.

Two-host placement

Fill Host A → the next VM lands on Host B.

Replay the worked example: weighted score picks A, memory-only picks B.

10 concurrent creates: no over-reservation.

Nothing fits → clear error, no partial state.

Failure injection

`kill -9` a VMM → auto-heal.

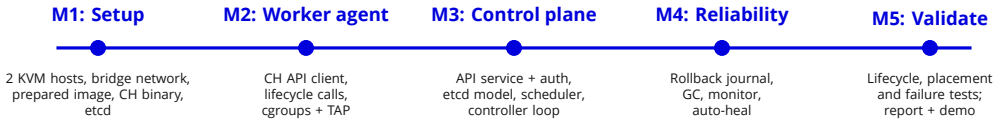
Kill agent mid-create → rollback on restart.

Cut Host B network → NotReady → reschedule.

Bad cert / replayed request → `401`.

Leak check after every test: no stray `cloud-hypervisor` PIDs, sockets, TAP devices or cgroups, and `MemAvailable` returns to its baseline.

Deployment Timeline



Method: build bottom-up – the agent works against Cloud Hypervisor on its own before the controller exists. Each milestone ends with its tests passing on both hosts.

Future scope: proxy for VM service networking, 3-node etcd for controller HA, Prometheus metrics, more than two hosts.

Bibliography

[1] Cloud Hypervisor Project. *Cloud Hypervisor API*. github.com/cloud-hypervisor/cloud-hypervisor/blob/main/docs/api.md

[2] The Kubernetes Authors. *Kubernetes Components*. kubernetes.io/docs/concepts/overview/components

[3] The Kubernetes Authors. *Scheduling Framework* and *Resource Bin Packing*. kubernetes.io/docs/concepts/scheduling-eviction

[4] D. Ongaro, J. Ousterhout. “In Search of an Understandable Consensus Algorithm.” USENIX ATC, 2014; and etcd documentation, etcd.io/docs

[5] A. Agache et al. “Firecracker: Lightweight Virtualization for Serverless Applications.” USENIX NSDI, 2020.

[6] H. Garcia-Molina, K. Salem. “Sagas.” ACM SIGMOD, 1987.

[7] E. Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, 2018.

[8] M. Jones, J. Bradley, N. Sakimura. *JSON Web Token (JWT)*. RFC 7519, 2015.

[9] H. Krawczyk, M. Bellare, R. Canetti. *HMAC: Keyed-Hashing for Message Authentication*. RFC 2104, 1997.

[10] R. Fielding, M. Nottingham, J. Reschke. *HTTP Semantics*. RFC 9110, 2022.

[11] T. Heo. *Control Group v2*. Linux kernel documentation, docs.kernel.org/admin-guide/cgroup-v2.html

[12] A. Kivity et al. “kvm: the Linux Virtual Machine Monitor.” Ottawa Linux Symposium, 2007.

Thank You for Your Attention!

Questions?